

INTERNSHIP PROJECT REPORT

NotesKeeper Application using JWT Authentication & Protected Routes

Task 3 – Authentication & Protected Routes (JWT)

Submitted by:

Srujan Tuppad

MERN Stack Development Internship Programme

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me during the course of this internship. This project, undertaken as Task 3 of my internship program, gave me the opportunity to apply my understanding of full-stack web development to a real, working application, and I am thankful for the guidance and encouragement I received along the way.

I am grateful to my internship mentor and reviewers for their constant support, for patiently answering my questions, and for pointing me toward better practices whenever my implementation needed improvement. Their feedback on the authentication module in particular helped me understand not just how to make the feature work, but why each security decision mattered.

I would also like to thank my institution, [College / University Name], and the faculty of the [Department Name] for providing the foundation of knowledge in web technologies and software engineering that made this internship possible. Finally, I extend my thanks to my family and friends for their patience and encouragement throughout this project.

This internship has been a valuable learning experience, and I am thankful for every opportunity it gave me to grow as a developer.

Abstract

This report documents the design and development of NotesKeeper, a full-stack web application that allows registered users to securely create, view, edit, and delete personal notes. The project was carried out as Task 3 of an internship program, with a specific focus on implementing user authentication and protecting application routes using JSON Web Tokens (JWT).

The application follows a client-server architecture. The frontend is built with React.js and Vite, offering a fast development workflow and a responsive user interface, while the backend is implemented using Node.js and Express.js to expose a RESTful API. MongoDB serves as the persistence layer, storing user credentials and note data through Mongoose schemas. User passwords are never stored in plain text; instead, they are hashed using bcrypt before being saved to the database, and every request to a protected endpoint is verified using a JSON Web Token issued at login.

The core of this task involved building a signup and login flow, issuing and verifying JWTs, protecting both frontend routes and backend API endpoints from unauthorized access, and allowing authenticated users to perform full CRUD (Create, Read, Update, Delete) operations on their own notes. React Router DOM was used on the client side to guard routes so that only logged-in users could reach the notes dashboard, while Express middleware performed the equivalent check on the server side for every incoming request.

This report walks through the objectives, technology stack, system architecture, folder structure, and implementation details of the project, supported by code snippets and screenshots taken from the working application. It concludes with a discussion of the challenges faced during development, the key learning outcomes from the internship task, and possible directions for extending the application in the future.

Table of Contents

Acknowledgement.....	i
Abstract	ii
Table of Contents.....	iii
1. Introduction	1
2. Objectives	1
3. Problem Statement.....	2
4. Technology Stack	2
4.1 Frontend Technologies.....	3
4.2 Backend Technologies.....	4
4.3 Security Libraries.....	4
5. System Architecture.....	4
6. Project Folder Structure	5
6.1 VS Code Project Explorer.....	6
7. Backend Implementation.....	7
7.1 Server Entry Point	7
7.2 Database Connection	8
7.3 User Model	8
7.4 Note Model	9
7.5 Authentication Controller	9
7.6 JWT Verification Middleware	11
7.7 Note Routes	12
7.8 Backend Server Running	12
8. Frontend Implementation.....	13
8.1 Frontend Server (Vite Development Server)	13
8.2 Axios Instance	13
8.3 Authentication Context.....	14
8.4 Application Routing.....	14
9. Authentication Flow (Signup, Login, JWT, Protected Routes, Logout).....	16
9.1 Signup	17
9.2 Login	18
9.3 Protected Routes	19
9.4 Logout.....	19
10. CRUD Operations (Create, Read, Update, Delete Notes).....	19

10.1 Note Controller	20
10.2 Read — Home Page / Note Listing.....	21
10.3 Create.....	21
10.4 Update	22
10.5 Delete.....	22
11. Security Features	23
11.1 Password Hashing	23
11.2 JWT Authentication.....	23
11.3 Protected APIs.....	23
11.4 Additional Considerations.....	24
12. Challenges Faced.....	24
12.1 Managing the JWT on the Frontend	24
12.2 Keeping Frontend and Backend Protection in Sync	24
12.3 Handling Token Expiry Gracefully.....	24
12.4 Password Hashing Timing.....	24
12.5 CORS and Local Development Setup	25
13. Learning Outcomes	25
14. Future Enhancements	25
15. Conclusion.....	26
16. References	Error! Bookmark not defined.

1. Introduction

Note-taking applications are among the most common examples used to teach and demonstrate full-stack web development, because they touch nearly every concept a web developer needs to master: user interfaces, client-server communication, databases, and — critically — user authentication. This report describes NotesKeeper, a note-taking web application built during an internship program, with Task 3 of the internship focused specifically on securing the application through JWT-based authentication and protected routing.

Before this task, a basic version of the notes application already existed with the ability to create and manage notes. However, the application had no concept of individual users; anyone could access and modify any note. Task 3 addressed this gap by introducing a proper signup and login system, so that each user's notes are private to their own account and can only be accessed after successful authentication.

The application is built using the MERN-adjacent stack — MongoDB, Express.js, React.js, and Node.js — with Vite used as the frontend build tool in place of Create React App for faster builds and a better development experience. Authentication is handled through JSON Web Tokens, which are compact, self-contained tokens issued by the server after a successful login and subsequently sent by the client with every request to a protected route.

This report is organised to first explain the objectives and problem the project sets out to solve, then walk through the technologies used, the architecture of the system, and the folder structure of the codebase. It then goes into the implementation details of both the backend and frontend, with particular emphasis on the authentication flow, before covering the CRUD functionality for notes, the security measures put in place, and the challenges faced during development. The report closes with the learning outcomes of the task, ideas for future enhancement, and a conclusion.

2. Objectives

The objectives set out for Task 3 of the internship were as follows:

- To design and implement a secure user registration (signup) system that stores user credentials safely in the database.
- To implement a login system that verifies user credentials and issues a JSON Web Token (JWT) upon successful authentication.
- To protect backend API routes so that note-related endpoints can only be accessed by authenticated users with a valid token.

- To protect frontend routes using React Router DOM, so that unauthenticated users attempting to reach the notes dashboard are redirected to the login page.
- To hash user passwords using bcrypt before storing them, ensuring that plain-text passwords are never persisted in the database.
- To implement complete CRUD (Create, Read, Update, Delete) functionality for notes, scoped to the logged-in user.
- To implement a logout mechanism that clears the stored authentication token and returns the user to a logged-out state.
- To test and demonstrate the authentication and protected-route behaviour with real, working screenshots of the application.

3. Problem Statement

A note-taking application that has no authentication layer is, in practice, not usable by more than one person. Without a way to identify who is using the application, there is no way to keep one user's notes separate from another's, and there is no way to prevent an unauthorised visitor from viewing, editing, or deleting notes that do not belong to them. This is the exact problem that Task 3 was designed to solve.

Specifically, the project needed to address the following concerns:

- Identity: the system needs a reliable way to know which user is making a request.
- Credential safety: passwords must never be stored or transmitted in a way that exposes them if the database or network traffic is compromised.
- Session handling without server-side session storage: the application needed a stateless way to keep users logged in across requests, which is where JWT fits in.
- Access control: both the user interface (frontend routes) and the underlying data (backend API) needed to be protected, since securing only one side of the application is not sufficient.
- Usability: the added security should not make the application difficult or confusing to use — signup, login, and logout needed to remain simple, quick actions for the end user.

The solution implemented in this project addresses each of these concerns using industry-standard tools: bcrypt for password hashing, JWT for stateless authentication, Express middleware for backend route protection, and React Router DOM's routing capabilities for frontend route protection.

4. Technology Stack

The NotesKeeper application was built using a combination of modern JavaScript technologies on both the frontend and backend, chosen for their popularity, strong community support, and suitability for building secure, full-stack web applications quickly. The table below summarises the stack, followed by a short discussion of the role each technology plays in the project.

Technology	Role in the Project
React.js	JavaScript library used to build the component-based user interface of the application.
Vite	Frontend build tool and development server, used in place of Create React App for faster startup and hot module reloading.
React Router DOM	Handles client-side routing, including redirecting unauthenticated users away from protected pages.
Axios	Promise-based HTTP client used by the frontend to communicate with the Express REST API.
Node.js	JavaScript runtime that powers the backend server.
Express.js	Minimal web framework used to build the REST API and define routes and middleware.
MongoDB	NoSQL document database used to store user accounts and note documents.
Mongoose	Object Data Modelling (ODM) library used to define schemas and interact with MongoDB from Node.js.
JWT (jsonwebtoken)	Used to generate and verify JSON Web Tokens for stateless authentication.
bcrypt	Used to hash user passwords before storing them, and to compare hashed passwords during login.

4.1 Frontend Technologies

React.js was chosen for the user interface because of its component-based structure, which made it straightforward to break the application down into reusable pieces such as the note card, the navigation bar, and the login and signup forms. Vite was used as the build tool because of its fast cold-start time and instant hot module replacement, which made the development cycle noticeably quicker than tools like Create React App, especially when repeatedly testing changes to the authentication flow. React Router DOM manages navigation between pages such as Login, Signup, Home, and Create Note, and was also used to implement the protected route wrapper that redirects unauthenticated users to the login page. Axios was used for all HTTP communication with the backend, including attaching the JWT to the Authorization header of protected requests.

4.2 Backend Technologies

The backend is built on Node.js and Express.js, which together provide a lightweight and flexible way to define REST API endpoints. Express middleware plays a central role in this project: a custom middleware function verifies the JWT on every request to a protected route before allowing the request to reach its controller logic. Mongoose was used to define schemas for the User and Note models and to interact with MongoDB in a structured, validated way, rather than working with raw MongoDB driver calls.

4.3 Security Libraries

Two libraries were central to the security of the application. The jsonwebtoken package was used to sign a token containing the user's identity at login and to verify that token on subsequent requests. The bcrypt package was used to hash passwords with a salt before storing them in MongoDB, and to safely compare a submitted password against the stored hash during login, without ever needing to store or compare plain-text passwords.

5. System Architecture

NotesKeeper follows a classic three-tier architecture, separating the application into a presentation tier, an application (logic) tier, and a data tier. This separation keeps the codebase organised and makes it possible to reason about the authentication flow independently from the note-management logic.

The presentation tier is the React.js single-page application running in the user's browser. It is responsible for rendering the login, signup, home, and note-editing screens, and for making HTTP requests to the backend using Axios. The application tier is the Express.js server, which exposes REST endpoints under `/api/auth` and `/api/notes`, applies the JWT verification middleware to protected routes, and contains the business logic for authentication and note management. The data tier is MongoDB, accessed through Mongoose, which stores two collections: one for user accounts and one for notes.

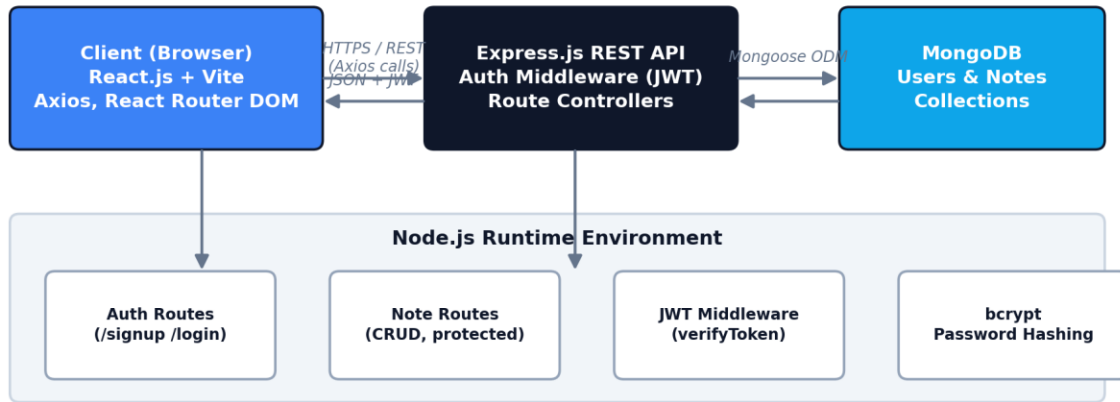


Figure: Three-tier architecture of the NotesKeeper application
(Presentation Tier → Application Tier → Data Tier)

Figure 1: Three-tier architecture of the NotesKeeper application.

When a user interacts with the application, requests flow from the browser to the Express API over HTTPS as JSON payloads. For protected endpoints, the request must include an Authorization header carrying the JWT issued at login. The verifyToken middleware intercepts the request, checks the token's signature and expiry, and either allows the request to proceed to its route handler or responds with a 401 Unauthorized error. Once a request is authorised, the corresponding controller communicates with MongoDB through Mongoose to read or write the required data, and the result is returned to the client as JSON.

6. Project Folder Structure

The project is organised as two separate folders — a backend folder and a frontend folder — each with its own package.json and dependencies. This separation keeps the client and server codebases independent, which also makes it possible to deploy them separately if required. The overall structure of the project is shown below.

```

notes-keeper/
|-- backend/
|   |-- config/           |-- db.js           # MongoDB connection setup
|   |-- models/
|   |   |-- User.js       # Mongoose schema for users
  
```

```

| | `-- Note.js          # Mongoose schema for notes
| |-- routes/
| | |-- authRoutes.js    # /api/auth/signup, /api/auth/login
| | |-- noteRoutes.js    # /api/notes CRUD routes (protected)
| |-- middleware/
| | |-- verifyToken.js   # JWT verification middleware
| |-- controllers/
| | |-- authController.js # signup/login logic
| | |-- noteController.js # note CRUD logic
| |-- .env               # environment variables (PORT, JWT_SECRET, MONGO_URI)
| |-- server.js          # Express app entry point
| |-- package.json
|
|-- frontend/
| |-- src/
| | |-- components/
| | | |-- Navbar.jsx
| | | |-- NoteCard.jsx
| | | |-- ProtectedRoute.jsx
| | |-- pages/
| | | |-- Login.jsx
| | | |-- Signup.jsx
| | | |-- Home.jsx
| | | |-- CreateNote.jsx
| | | |-- EditNote.jsx
| | |-- context/
| | | |-- AuthContext.jsx # stores auth state & token
| | |-- api/
| | | |-- axiosInstance.js # Axios instance with auth header
| | |-- App.jsx
| | |-- main.jsx
| |-- index.html
| |-- vite.config.js
| |-- package.json

```

Figure 2: Folder structure of the NotesKeeper backend and frontend.

6.1 VS Code Project Explorer

The screenshot below shows the actual project as opened in Visual Studio Code, confirming the folder and file organisation described above.

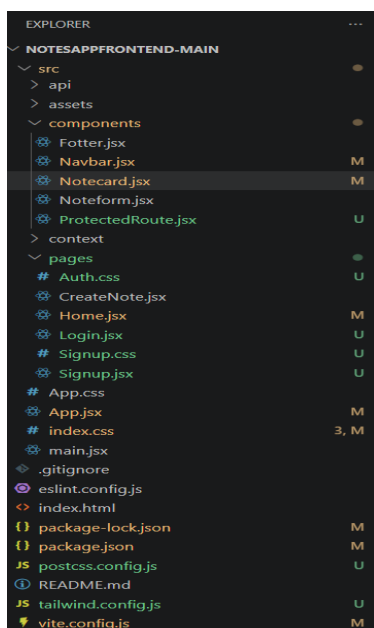


Figure 3: VS Code project explorer showing the NotesKeeper folder structure.

7. Backend Implementation

The backend is a Node.js application built with Express.js. It exposes a REST API consumed by the React frontend, connects to MongoDB using Mongoose, and is responsible for all authentication and data-persistence logic. This section walks through the main building blocks of the backend.

7.1 Server Entry Point

The server.js file is the entry point of the backend application. It initialises Express, applies middleware such as cors and express.json, connects to MongoDB, and mounts the authentication and note routers under their respective base paths.

```
// server.js
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const authRoutes = require('./routes/authRoutes');
const noteRoutes = require('./routes/noteRoutes');

dotenv.config();
connectDB();

const app = express();
```

```
app.use(cors());
app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/notes', noteRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

7.2 Database Connection

The `db.js` file inside the `config` folder encapsulates the MongoDB connection logic using Mongoose, keeping the connection string and connection options out of `server.js` and easy to reuse or modify.

```
// config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected successfully');
  } catch (error) {
    console.error('MongoDB connection failed:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

7.3 User Model

The User schema stores the account name, a unique email address, and the bcrypt-hashed password. The unique constraint on email is enforced at the database level to prevent duplicate accounts.

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true }, // stores bcrypt hash
  },
```

```
    { timestamps: true }  
  );  
  
  module.exports = mongoose.model('User', userSchema);
```

7.4 Note Model

Each note document stores a title, content, and a reference to the user it belongs to. This user reference is what makes it possible to scope CRUD operations to the currently logged-in user.

```
import mongoose from "mongoose"  
const noteSchema=new mongoose.Schema({  
  title:{  
    type:String,  
    required:true,  
    trim:true  
  },  
  content:{  
    type:String,  
    required:true,  
  
  }  
},  
{  
  timestamps:true,  
})  
const Note=mongoose.model("Note",noteSchema);  
export default Note;
```

7.5 Authentication Controller

The authController.js file contains the signup and login logic. During signup, the password is hashed with bcrypt before the user document is saved. During login, the submitted password is compared against the stored hash, and on success, a JWT is signed and returned to the client.

```
import User from "../models/user.model.js";  
import bcrypt from "bcryptjs";  
import jwt from "jsonwebtoken";  
  
// Register User  
export const registerUser = async (req, res) => {  
  try {  
    const { name, email, password } = req.body;
```

```
const existingUser = await User.findOne({ email });

if (existingUser) {
  return res.status(400).json({
    message: "User already exists",
  });
}

const hashedPassword = await bcrypt.hash(password, 10);

const user = await User.create({
  name,
  email,
  password: hashedPassword,
});

res.status(201).json({
  message: "User registered successfully",
  user,
});
} catch (error) {
  res.status(500).json({
    message: error.message,
  });
}
};

// Login User
export const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });

    if (!user) {
      return res.status(400).json({
        message: "Invalid credentials",
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(400).json({
        message: "Invalid credentials",
      });
    }

    const token = jwt.sign(
```

```
    { id: user._id },
    "mysecretkey",
    { expiresIn: "1d" }
  );

  res.status(200).json({
    message: "Login successful",
    token,
  });
} catch (error) {
  res.status(500).json({
    message: error.message,
  });
}
};
export const getProfile = async (req, res) => {
  res.status(200).json({
    message: "Protected Route Accessed",
    user: req.user
  });
};
};
```

7.6 JWT Verification Middleware

The `verifyToken` middleware reads the `Authorization` header, extracts the token, and verifies it using the secret key. If verification succeeds, the decoded user information is attached to the request object so that downstream controllers know which user is making the request. If verification fails or the header is missing, the middleware responds with a 401 status and the request never reaches the note controller.

```
import jwt from "jsonwebtoken";

const verifyToken = (req, res, next) => {
  try {
    const authHeader = req.header("Authorization");

    if (!authHeader) {
      return res.status(401).json({
        message: "Access Denied",
      });
    }

    const token = authHeader.replace("Bearer ", "");

    const verified = jwt.verify(token, "mysecretkey");

    req.user = verified;
  }
};
```



```
    next();
  } catch (error) {
    res.status(401).json({
      message: "Invalid Token",
    });
  }
};

export default verifyToken;
```

7.7 Note Routes

The note routes apply the `verifyToken` middleware to every endpoint, ensuring that CRUD operations on notes are only possible for authenticated requests.

```
import express from "express";
import { createNote, deleteNote, getNotes, updateNote }
from "../Controllers/note.controller.js";

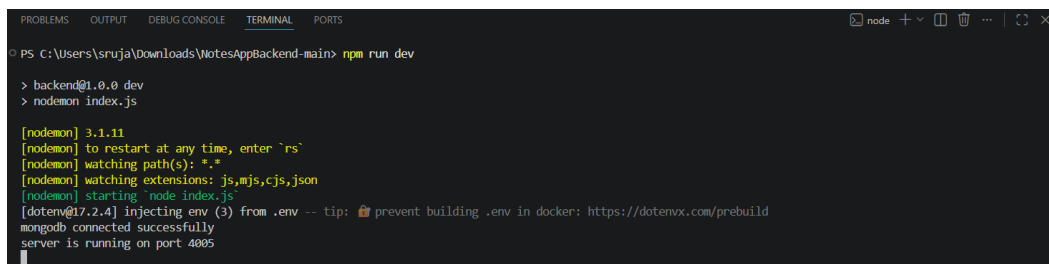
const router=express.Router();

router.post("/create-note",createNote)
router.get("/get-notes",getNotes)
router.put("/update-note/:id",updateNote)
router.delete("/delete-note/:id",deleteNote)

export default router;
```

7.8 Backend Server Running

The screenshot below shows the backend terminal after starting the Express server with the MongoDB connection established, confirming that the backend is running and ready to accept requests.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sruja\Downloads\NotesAppBackend-main> npm run dev

> backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): "."
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting node index.js
[dotenv@17.2.4] injecting env (3) from .env -- tip: 📖 prevent building .env in docker: https://dotenvx.com/prebuild
mongodb connected successfully
server is running on port 4805
```

Figure 4: Backend terminal showing the Express server running and connected to MongoDB.

8. Frontend Implementation

The frontend is a single-page application built with React.js and bundled using Vite. It is organised into pages (Login, Signup, Home, Create Note, Edit Note), reusable components (Navbar, NoteCard, ProtectedRoute), and a small authentication context that keeps track of the logged-in user and their token across the application.

8.1 Frontend Server (Vite Development Server)

During development, the frontend is served using the Vite development server, which provides fast reloads whenever a file is saved. The screenshot below shows the terminal output after starting the frontend with `npm run dev`.

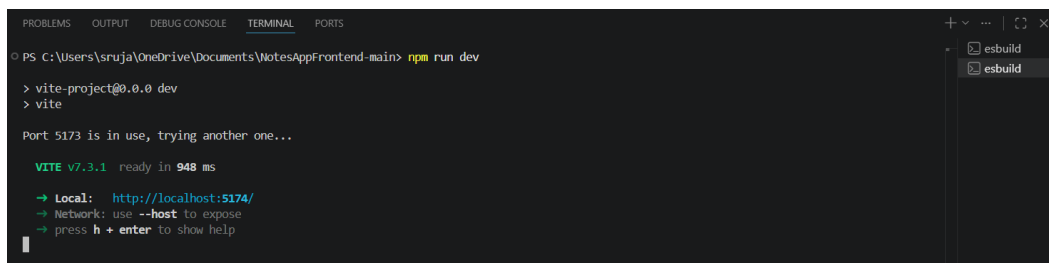


Figure 5: Frontend terminal showing the Vite development server running.

8.2 Axios Instance

A single Axios instance is configured for the whole application. Before every request, an interceptor reads the JWT from local storage and, if present, attaches it to the Authorization header, so that individual components do not need to repeat this logic.

```
// api/axiosInstance.js
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000/api',
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

```
export default api;
```

8.3 Authentication Context

The AuthContext keeps the current user and token in React state, exposes login and logout functions, and persists the token to local storage so the user stays logged in across page refreshes.

```
// context/AuthContext.jsx
import { createContext, useState } from 'react';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [token, setToken] = useState(localStorage.getItem('token'));

  const login = (newToken) => {
    localStorage.setItem('token', newToken);
    setToken(newToken);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
  };

  return (
    <AuthContext.Provider value={{ token, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

8.4 Application Routing

App.jsx defines the application's routes using React Router DOM. The notes dashboard and note-editing routes are wrapped in a ProtectedRoute component, so that any attempt to visit them without a valid token results in an automatic redirect to the login page.

```
import React from "react";
import Home from "../pages/Home";
import CreateNote from "../pages/CreateNote";
import Login from "../pages/Login";
import Signup from "../pages/Signup";
```

```

import { Routes, Route, useLocation } from "react-router-dom";

import Navbar from "../components/Navbar";
import Footer from "../components/Footer";
import ProtectedRoute from "../components/ProtectedRoute";

const App = () => {
  const location = useLocation();

  const hideLayout =
    location.pathname === "/login" ||
    location.pathname === "/signup";

  return (
    <>
      {!hideLayout && <Navbar />}

      <main
        className={
          hideLayout
            ? ""
            : "container mx-auto px-4 py-6"
        }
      >
        <Routes>

          {/* Public Routes */}

          <Route path="/login" element={<Login />} />

          <Route path="/signup" element={<Signup />} />

          {/* Protected Routes */}

          <Route
            path="/"
            element={
              <ProtectedRoute>
                <Home />
              </ProtectedRoute>
            }
          />

          <Route
            path="/create"
            element={
              <ProtectedRoute>
                <CreateNote />
              </ProtectedRoute>
            }
          />
        </Routes>
      </>
    )
  };

```

```

    />

    </Routes>
  </main>

  {!hideLayout && <Footer />}
</>
);
};

export default App;

```

9. Authentication Flow (Signup, Login, JWT, Protected Routes, Logout)

This section describes, end-to-end, how a user moves from an anonymous visitor to an authenticated user of NotesKeeper, and how the application enforces that only authenticated users can reach the notes dashboard. The sequence diagram below summarises the signup and login exchanges between the browser, the Express API, and MongoDB.

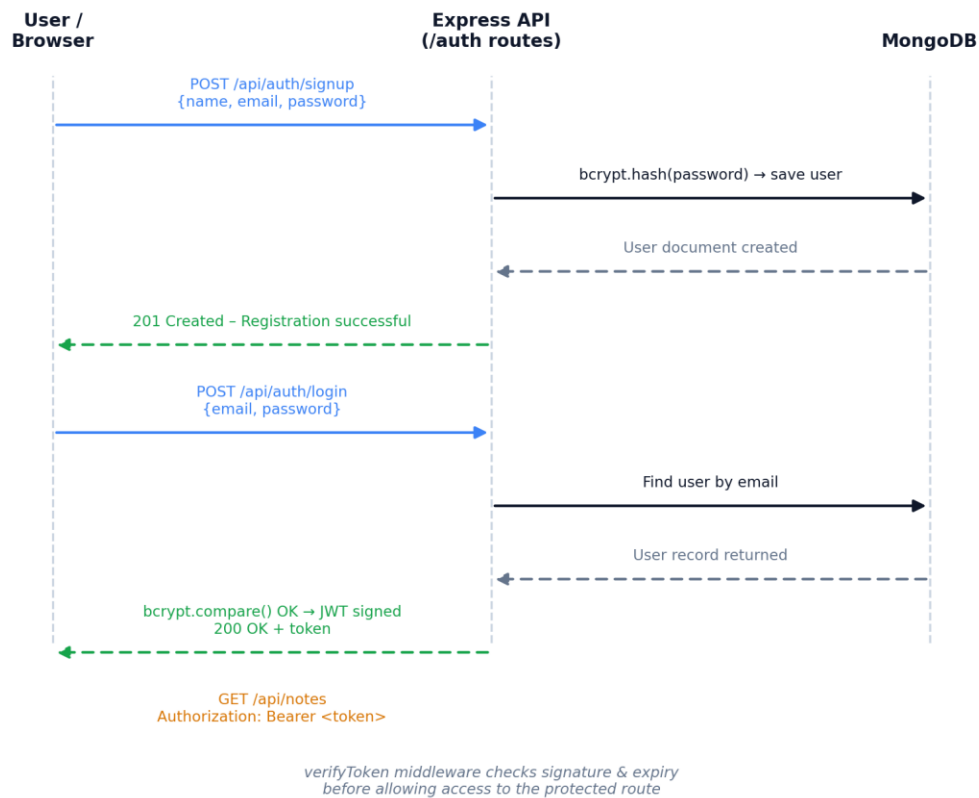


Figure 6: Sequence of requests exchanged during signup and login, leading up to an authenticated request.

9.1 Signup

On the Signup page, the user provides their name, email address, and password. When the form is submitted, the frontend sends a POST request to `/api/auth/signup`. On the backend, the `authController` checks whether a user with that email already exists; if not, the password is hashed using `bcrypt.hash()` with a salt round of 10, and a new user document is created in MongoDB. The server responds with a success message and the application then redirects the user to the login page.

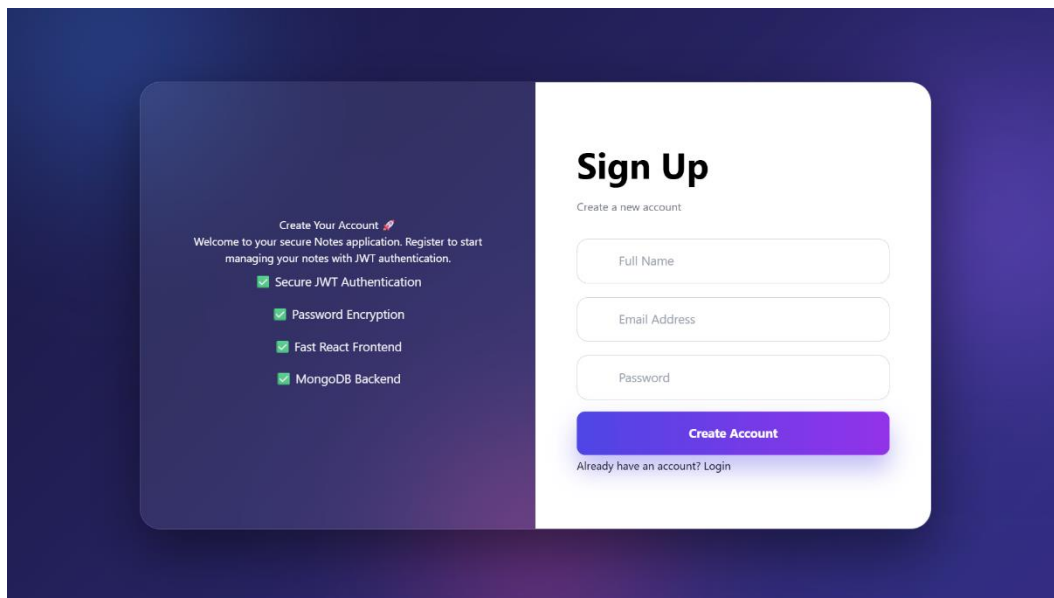


Figure 7: Signup page where a new user enters their name, email, and password.

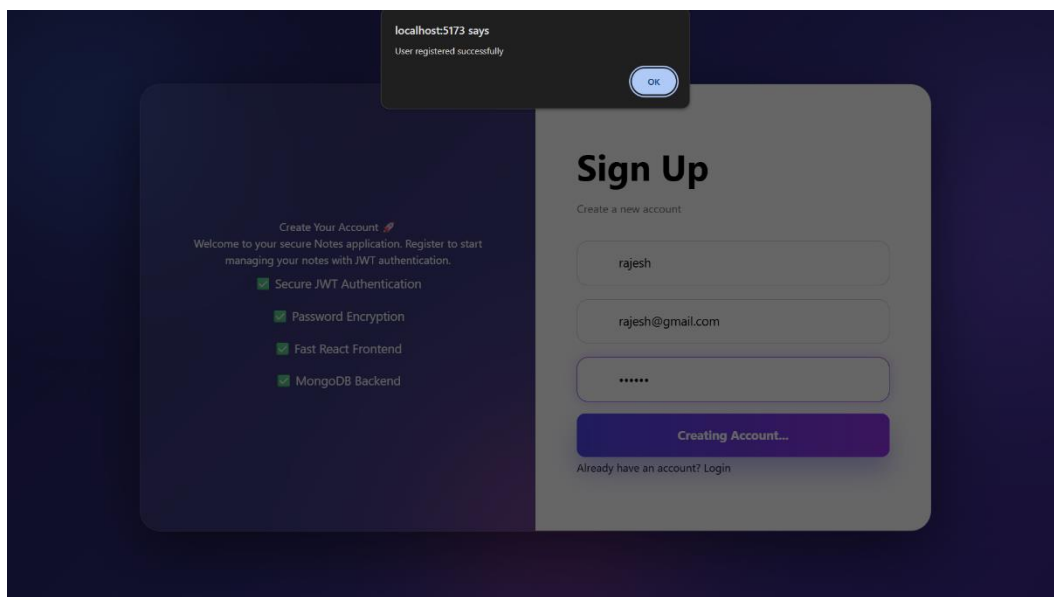


Figure 8: Confirmation shown after a successful registration.

9.2 Login

On the Login page, the user enters their registered email and password. The frontend sends this data to `/api/auth/login`. The backend looks up the user by email and uses `bcrypt.compare()` to check the submitted password against the stored hash. If the credentials are valid, the server signs a JWT containing the user's id and name, using a secret key stored in the environment variables, and returns this token to the client along with basic user details. The frontend stores the token in local storage through the `AuthContext` and redirects the user to the home page.

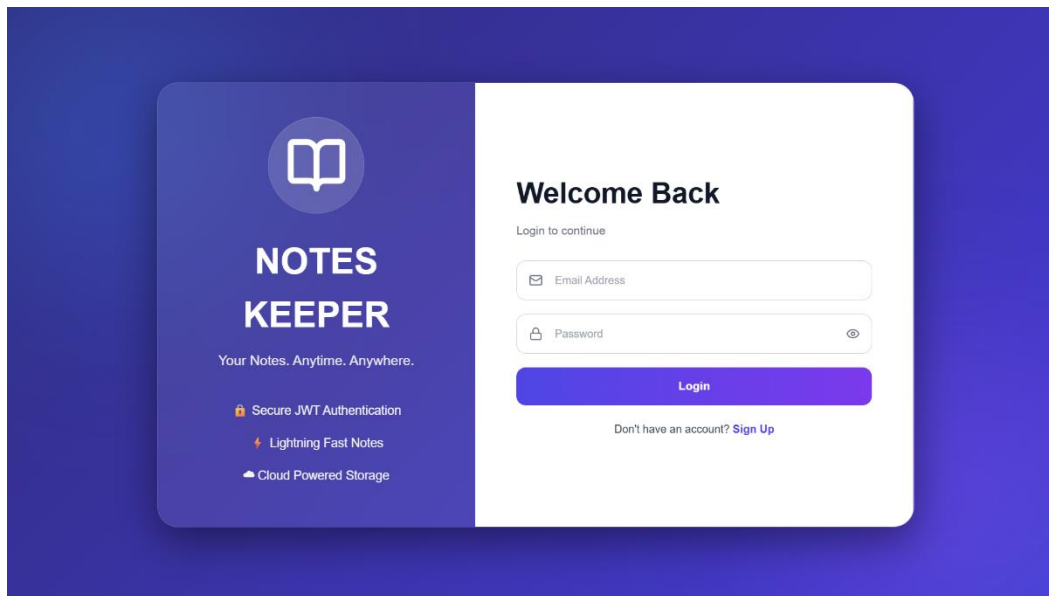


Figure 9: Login page where a registered user enters their email and password.

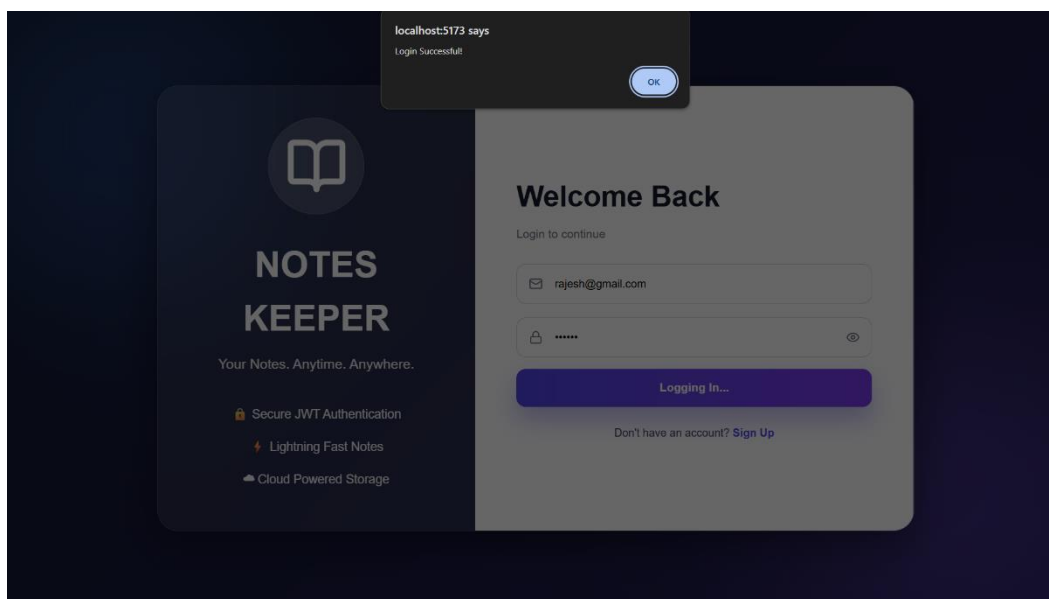


Figure 10: Home page shown immediately after a successful login.

9.3 Protected Routes

Two layers of protection are used in this project. On the frontend, the `ProtectedRoute` component checks whether a token is present in the `AuthContext`. If no token is found, the component redirects the visitor to `/login` instead of rendering the requested page, using React Router DOM's `Navigate` component. This prevents an unauthenticated visitor from seeing the notes dashboard even briefly.

```
import React from "react";
import { Navigate } from "react-router-dom";

const ProtectedRoute = ({ children }) => {
  const token = localStorage.getItem("token");

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

On the backend, the same idea is enforced by the `verifyToken` middleware described in Section 7.6, which is attached to every note-related route. Even if a request bypassed the frontend entirely — for example, by calling the API directly with a tool such as Postman — it would still be rejected with a 401 Unauthorized response unless a valid token is supplied. This dual protection is important: frontend route guarding improves the user experience, while backend middleware is what actually keeps the data secure.

9.4 Logout

Logging out is handled by the `logout` function exposed through `AuthContext`. It removes the token from local storage and clears it from application state, which immediately causes `ProtectedRoute` to treat the user as unauthenticated on the next navigation, sending them back to the login page. Because JWTs are stateless, no server-side action is required to invalidate the session; the token simply stops being sent by the client once it is removed.

10. CRUD Operations (Create, Read, Update, Delete Notes)

Once a user is authenticated, they can manage their own notes through a full set of CRUD operations. Every request described in this section requires a valid JWT, and every note document is linked to the user who created it through the `user` field on the `Note` model, so that one user's notes are never mixed with another's.

10.1 Note Controller

The `noteController.js` file implements the four CRUD operations. Each function reads `req.user.id`, populated by the `verifyToken` middleware, and uses it to scope the operation to the correct user.

```
import Note from "../models/note.models.js";
export const createNote=async(req,res)=>{

  try{
    const {title,content}=req.body;
    if(!title || !content){
      return res.status(400).json({
        success:false,
        message:"All field are required"
      });
    }
    const newNote=new Note({title,content})
    await newNote.save();
    res.status(201).json(newNote);
    console.log("note craeted")

  }
  catch(error){
    res.status(500).json({message:"Internal server error"});
  }
}
export const getNotes=async(req,res)=>{
  try{
    const notes=await Note.find().sort({createdAt:-1});
    res.status(200).json(notes);

  }
  catch(err){
    res.status(500).json({message:err.message});
  }
}
export const updateNote=async(req,res)=>{
  try{
    const {title,content}=req.body;
    const upadtedNote=await
Note.findByIdAndUpdate(req.params.id,{title,content},{new:true});
    if(!updateNote){
      return res.status(404).json({message:"Note not updated"})
    }
    res.status(200).json(updateNote)

  }
  catch(err){
    res.status(500).json({message:err.message});
  }
}
```

```

}
export const deleteNote=async(req,res)=>{
  try{
    const deletenote=await Note.findByIdAndDelete(req.params.id);
    if(!deletenote){
      res.status(404).json({message:"Note Not found"})
    }
    res.status(200).json({message:"Note deleted sucessfully"})
  }
  catch(err){
    res.status(500).json({message:err.message});
  }
}

```

10.2 Read — Home Page / Note Listing

After logging in, the user lands on the home page, which fetches all notes belonging to their account through a GET request to `/api/notes` and displays them as a list of note cards. Each card shows the note title, content preview, and the date it was created, along with Edit and Delete actions.

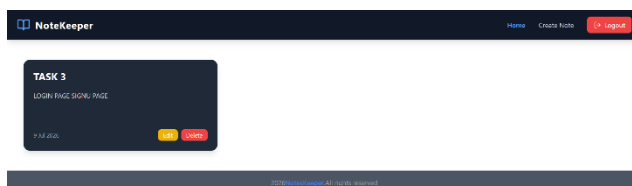


Figure 12: Home page listing the notes belonging to the logged-in user.

10.3 Create

The Create Note page provides a simple form with a title and content field. On submission, the frontend sends a POST request to `/api/notes` with the JWT attached, and the newly created note is returned and added to the list on the home page.

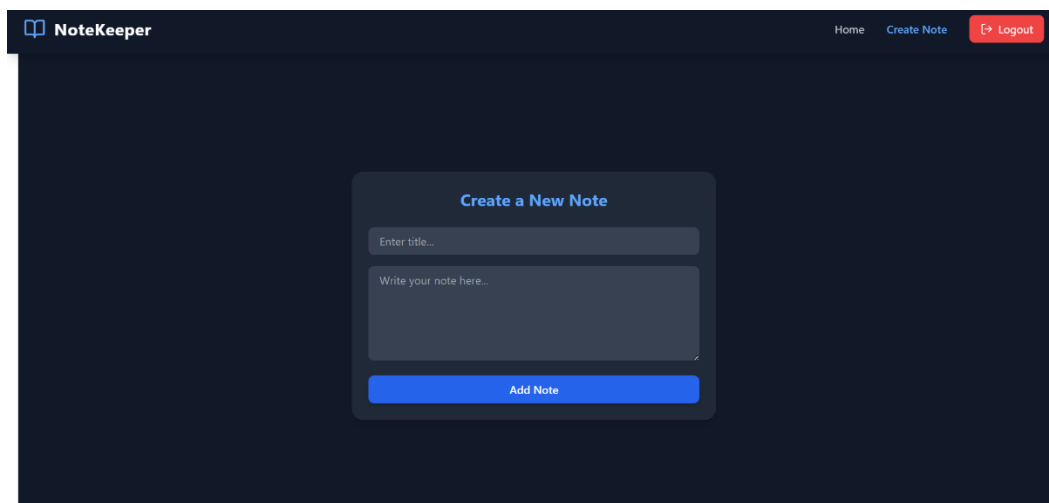
The screenshot shows the 'Create a New Note' page of the NotesKeeper application. The page has a dark blue background. At the top, there is a navigation bar with the 'NoteKeeper' logo on the left and links for 'Home', 'Create Note', and a red 'Logout' button on the right. In the center of the page, there is a light blue rounded rectangle containing the form. The form has a title 'Create a New Note' in bold. Below the title, there is a text input field labeled 'Enter title...' and a larger text area labeled 'Write your note here...'. At the bottom of the form, there is a blue button labeled 'Add Note'.

Figure 13: Create Note page used to add a new note.

10.4 Update

Clicking Edit on a note card takes the user to the Edit Note page, which pre-fills the form with the existing title and content. Submitting the form sends a PUT request to `/api/notes/:id`, and the note is updated in place, both in the database and in the list displayed on the home page.

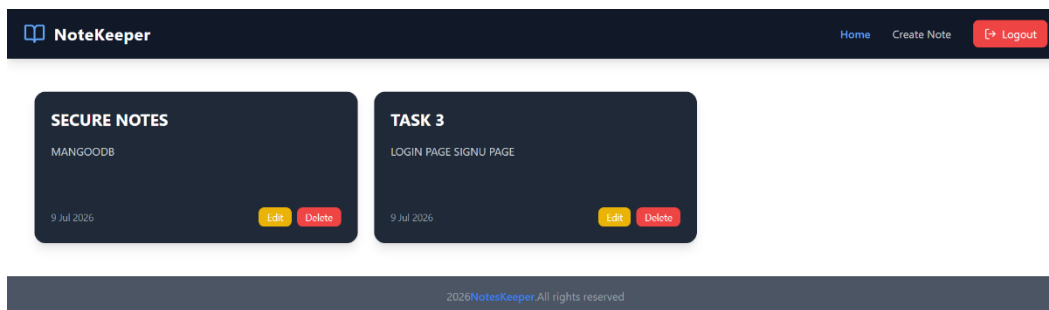
The screenshot shows the 'Edit Note' page of the NotesKeeper application. The page has a dark blue background. At the top, there is a navigation bar with the 'NoteKeeper' logo on the left and links for 'Home', 'Create Note', and a red 'Logout' button on the right. Below the navigation bar, there are two note cards. The first card is titled 'SECURE NOTES' and contains the text 'MANGOODB'. The second card is titled 'TASK 3' and contains the text 'LOGIN PAGE SIGNU PAGE'. Both cards show the date '9 Jul 2026' and have yellow 'Edit' and red 'Delete' buttons. At the bottom of the page, there is a footer with the text '2026NotesKeeper.All rights reserved'.

Figure 14: Edit Note page pre-filled with the note's existing content for editing.

10.5 Delete

Clicking Delete on a note card sends a DELETE request to `/api/notes/:id`. The backend confirms that the note belongs to the requesting user before deleting it, after which the note is removed from the displayed list.

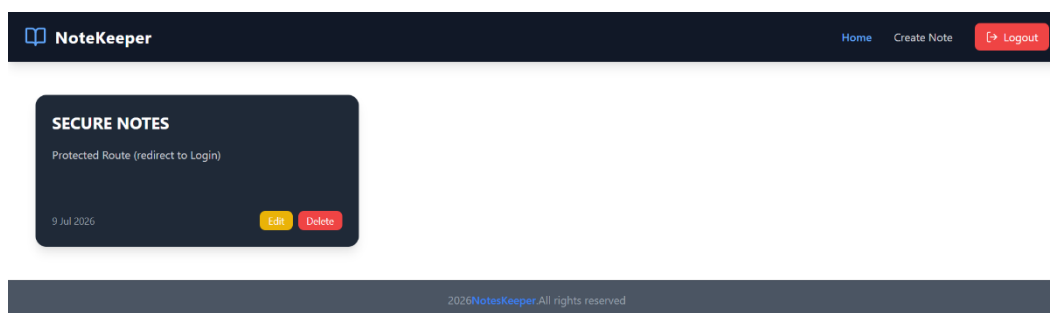


Figure 15: Delete confirmation for removing a note from the account.

11. Security Features

Security was the primary focus of Task 3, and several measures were put in place to keep user credentials and note data safe.

11.1 Password Hashing

User passwords are never stored in plain text. During signup, `bcrypt.hash()` generates a salted hash of the password with 10 salt rounds before it is saved to MongoDB. During login, `bcrypt.compare()` checks the submitted password against the stored hash without ever needing to reverse the hash, so the original password is never reconstructed or exposed, even to the application itself.

11.2 JWT Authentication

JSON Web Tokens provide a stateless way to keep users authenticated. Once issued at login, a token carries the user's id and name in its payload, signed with a secret key stored in an environment variable and set to expire after one day. Because the token is signed rather than encrypted, its payload is technically readable, but it cannot be modified without invalidating the signature, which the `verifyToken` middleware checks on every protected request.

11.3 Protected APIs

Every note-related endpoint on the backend is wrapped with the `verifyToken` middleware, so no note data can be created, read, updated, or deleted without a valid Authorization header. Combined with scoping

every query to `req.user.id`, this ensures that even an authenticated user cannot access or modify another user's notes by guessing note IDs.

11.4 Additional Considerations

- Environment variables (`.env`) are used to keep the JWT secret and MongoDB connection string out of the source code.
- CORS is configured on the Express server to control which origins may call the API.
- Mongoose schema validation (required fields, unique email) provides an additional layer of data integrity at the database level.

12. Challenges Faced

Building the authentication and protected-route functionality involved a number of practical challenges that took some trial and error to resolve.

12.1 Managing the JWT on the Frontend

Deciding where and how to store the JWT on the client side required some thought. Storing it in local storage made it simple to persist the session across page refreshes, but also meant being careful about when the token was read and attached to requests. Setting up the Axios interceptor to handle this centrally, rather than attaching the header manually in every API call, resolved most of the repetition and reduced the chance of forgetting to include the token on a protected request.

12.2 Keeping Frontend and Backend Protection in Sync

Early in development, the frontend route guarding was implemented before the backend middleware, which meant that although the UI correctly hid the notes dashboard from unauthenticated users, the API endpoints themselves were still open to direct requests. This highlighted the importance of treating backend protection as the real security boundary, with frontend route guarding as a usability layer on top of it rather than a substitute for it.

12.3 Handling Token Expiry Gracefully

Because the JWT is set to expire after one day, a request made with an expired token would fail with a 401 response. Initially, this simply showed a generic error to the user. This was refined so that a 401 response from any protected API call also triggers a logout on the frontend, clearing the stale token and redirecting the user to the login page, rather than leaving them on a broken page.

12.4 Password Hashing Timing

An early version of the signup logic accidentally hashed the password after the user document had already been saved, which meant the first attempt stored the plain-text password. Restructuring the controller so that hashing happens before the database call, and testing the stored value directly in MongoDB, confirmed that passwords were being hashed correctly before being persisted.

12.5 CORS and Local Development Setup

Since the frontend (running on the Vite development server) and the backend (running on Express) operate on different ports during local development, requests were initially blocked by the browser's CORS policy. Configuring the cors middleware on the Express server resolved this and allowed the two halves of the application to communicate correctly during development.

13. Learning Outcomes

This task strengthened both my practical and conceptual understanding of building secure web applications. The key takeaways from this internship task were:

- A clearer understanding of how JWT-based authentication works end-to-end, from signing a token at login to verifying it on every protected request.
- Hands-on experience with bcrypt for password hashing, and a better appreciation of why plain-text password storage is unacceptable in any real application.
- Practical experience designing Express middleware and understanding how middleware fits into the request-response cycle.
- A better understanding of protecting routes on both the frontend and backend, and why relying on frontend protection alone is not sufficient.
- Experience structuring a full-stack project into clean, maintainable folders, separating models, routes, controllers, and middleware on the backend, and pages, components, and context on the frontend.
- Improved debugging skills, particularly around diagnosing CORS issues, token expiry problems, and request/response flow between the frontend and backend.
- Familiarity with using React Context to manage authentication state across a React application without prop drilling.

14. Future Enhancements

While the current implementation satisfies the objectives of Task 3, there are several ways the application could be extended in the future:

- Refresh tokens: introducing a refresh-token mechanism so that users are not forced to log in again every time the access token expires.
- Password reset: adding a "forgot password" flow with email-based verification to let users reset a forgotten password.
- Input validation and error feedback: adding more detailed client-side validation and clearer error messages on the signup and login forms.
- Note search and tagging: allowing users to search their notes and organise them with tags or categories.
- Rich text or markdown support: allowing notes to include basic formatting rather than plain text only.
- Deployment: deploying the frontend and backend to a cloud platform with HTTPS enforced, and moving secrets to a managed environment configuration.
- Rate limiting: adding rate limiting on the login and signup endpoints to reduce the risk of brute-force attempts.

15. Conclusion

Task 3 of this internship focused on turning NotesKeeper from an open, unauthenticated notes application into a properly secured, multi-user system. By implementing signup and login flows backed by bcrypt password hashing, issuing and verifying JSON Web Tokens, and protecting both the frontend routes and backend API endpoints, the application now ensures that only authenticated users can create, view, edit, or delete their own notes.

Working through this task gave me practical, end-to-end experience with authentication in a full-stack JavaScript application — an area that is central to almost any real-world web application. Beyond the specific tools used here, the underlying ideas of never trusting the client, verifying every protected request on the server, and never storing sensitive data such as passwords in plain text are principles I will carry forward into future projects.

Overall, this internship task was a valuable, hands-on extension of my earlier coursework in web development, and it has left me more confident in my ability to design and build secure, production-style authentication systems.